Network Traffic Profiler Dashboard

Client - Plymouth City College [Tomek Bergier]

Group 15

Team Leader - Amelia Lee Sze Yai

(write your full name & role pls:))

Scrum Master - Timothy Birtles

Tomiris

Sofia

CONTENT PAGE

Network Traffic Profiler Dashboard **1**
Client - Plymouth City Council [Tomek Bergier] 1
Group 15 1
CONTENT PAGE 2
IMPORTANT NOTES 2
1) Introduction (Sofia , Amelia) 3
2) Project Management (All members) 3
Project planning material (content from semester 1) (Tomiris) 3
Sprint planning and execution (Tim) 3
Risk assessment and mitigation (Sofia) 3
Project schedule (Tomiris) 3
Client management (Amelia) 3
3) Detailed System Design (Tim & Tomiris) 4
4) Quality Assurance (Sofia & Amelia) 4
5) Conclusion (Tim, Tomiris) 4
6) References (3 References Each) 4
7) Appendix 4

IMPORTANT NOTES

- Each section/subsection should have the names of the member who wrote it.
- Try to distribute according to team contributions (equally if possible, but if not, the ones who did less technical work should contribute more here).
- Recommended page number is around 80 pages. Upper limit is 100 pages (penalty if crossing this). There is no minimum page limit. The key point is to be concise.

1) Introduction (Sofia , Amelia)

– introduce client, problem statement, brief system requirements and any other necessary details on the project

Client Introduction

The Network Traffic Profiler Dashboard project is being developed for the client, Plymouth City College, with Tomek Bergier as the client representative

Problem statement & Part of system requirements & Target Users

Modern network security tools are often complex, difficult to configure, and aimed at technical experts. Smaller organisations and non-specialist users may struggle to interpret raw PCAP data or understand network behaviour using existing tools.

This project addresses this gap by creating a lightweight, user-friendly dashboard that allows users to upload PCAP files and receive understandable insights into their network traffic.The system is designed to extract and process meaningful network flow features, apply machine learning to classify traffic activity and detect abnormal behaviour, and present results in an intuitive visual format.

The target users are non-technical staff from small and medium-sized enterprises (SMEs). By combining data processing, machine learning, and visualisation into a single workflow, the dashboard provides actionable insights without requiring advanced networking expertise.

2) Project Management (All members)

Project planning material (content from semester 1) (Tomiris)

Scope Definition and Requirements Formalization

At the start of Semester 1, the team established a clear project foundation through the creation of a detailed Requirements Document. The Requirements Document addressed both functional and non-functional requirements and acted as the primary guide for all design, development, and evaluation activities within the project lifecycle.

Functional requirements addressed the system’s functionality, while non-functional requirements addressed the system’s performance, usability, maintainability, and reliability, among other factors. The requirements were well-structured for better clarity, feasibility, and alignment with the client and the team.

The stability of the defined requirements within the project indicates the effectiveness of the initial scope definition and the alignment with the client and the team. As a result, no major scope of renegotiations or structural changes were required during development.

Work Breakdown and Task Structuring

Following requirement formalization, the project scope was decomposed into manageable and traceable work units. Requirements were broken down into implementable features (e.g., dashboard, data processing, machine learning), and these were further divided into weekly development tasks.

The backlog was structured hierarchically based on semesters and academic milestones. This breakdown ensured traceability of requirements, logical ordering of development activities, ownership of tasks, balancing of workloads, and incremental progress towards completion.

This breakdown served as a robust planning foundation before executing iterative sprints (refer to the following section on "Sprint planning and execution").

Governance Structure and Role Allocation

To ensure accountability and clarity of operations, formal roles were defined at the time of project initiation. Although collaboration was very integrated among team members, formal roles created a clear governance structure.

Strategic and Coordination Roles

- Product Owner (Sophia) – Responsible for requirement validation, task prioritization, decision-making, and monitoring overall project progress.
- Team Leader (Amelia) – Maintained workflow continuity, resolved bottlenecks, and supported coordination across both technical and organizational dimensions.
- Scrum Master (Tim) – Facilitated compliance with Agile principles and formalized processes in terms of structure and planning.
- Secretary (Tomiris) – Formalized meeting minutes and internal documentation records.

Although the formal roles were established, the responsibilities remained partially dynamic, because team members collaborated in different areas to encourage shared ownership and flexibility in execution of tasks.

Task allocation considered individual expertise and academic workload to ensure equitable contribution, with periodic oversight from the academic tutor.

Documentation and Version Control Strategy

To achieve transparency, traceability, and documentation management, the team used GitHub as a central repository to store source code, documentation, and proof of contribution. Version control helped to track all changes and maintain a clear development history throughout the project.

In addition, meeting minutes, design decisions, and planning documents were kept and managed using shared collaborative documents. These were later formalized and exported for submission consistency.

This approach to documentation provided clear visibility and traceability of the project development and was applicable to all team members.

Planning Oversight and Progress Monitoring
Throughout Semester 1, planning was centered around ensuring organizational and structural stability before embarking on intensive implementation.
Internal meetings were conducted for visibility of short-term progress and early identification of issues, while documented agendas and minutes ensured continuity and clarity in decision-making.

Sprint planning and execution (Tim)

To ensure our initial design requirements remained aligned with our client's requirements, the Agile methodology was adopted. This approach was selected for its emphasis on adaptive planning and capacity for rapid response to change (Kumar and Bhati, 2012).

Project management began with the creation of a comprehensive product backlog, which was split into Semester One and Semester Two milestones. We operated in biweekly sprints which provided an excellent balance between development, progress, and evaluation. During planning sessions, tasks were selected from the backlog and prioritized based on technical prerequisites and estimated completion time, ensuring the workload assigned to each cycle was realistic. It was important that the selection process be methodical as failure to implement certain functionalities would have stalled development down the line.

At the start of each sprint, we met to evaluate and refine the backlog, discussing each item and how it could be decomposed into smaller more manageable tasks. Tasks were then delegated to team members based on expertise and individual capacity, balancing the workload across the team by factoring in task density and estimated effort.

Twice-weekly standup meetings focused on tracking progress and identifying pitfalls, ensuring progress was not stalled by technical hurdles nor lack of communication and that team members remained aligned. To address previous over-running meetings, in semester two, the Scrum Master introduced structured agendas for each standup meeting. This ensured meetings adhered to the 15 minute timebox that standup meetings were intended to be (Laoyan, 2025) but also guaranteed all actionables from the previous meeting were reviewed preventing any tasks from slipping through the net.

Trello was utilised as the primary Kanban style project management tool, providing a single location to track the project status. The project's Trello board was organised into columns, with a column each for 'Backlog', 'In progress', and 'Completed', in addition to individual columns for each weekly Scrum.

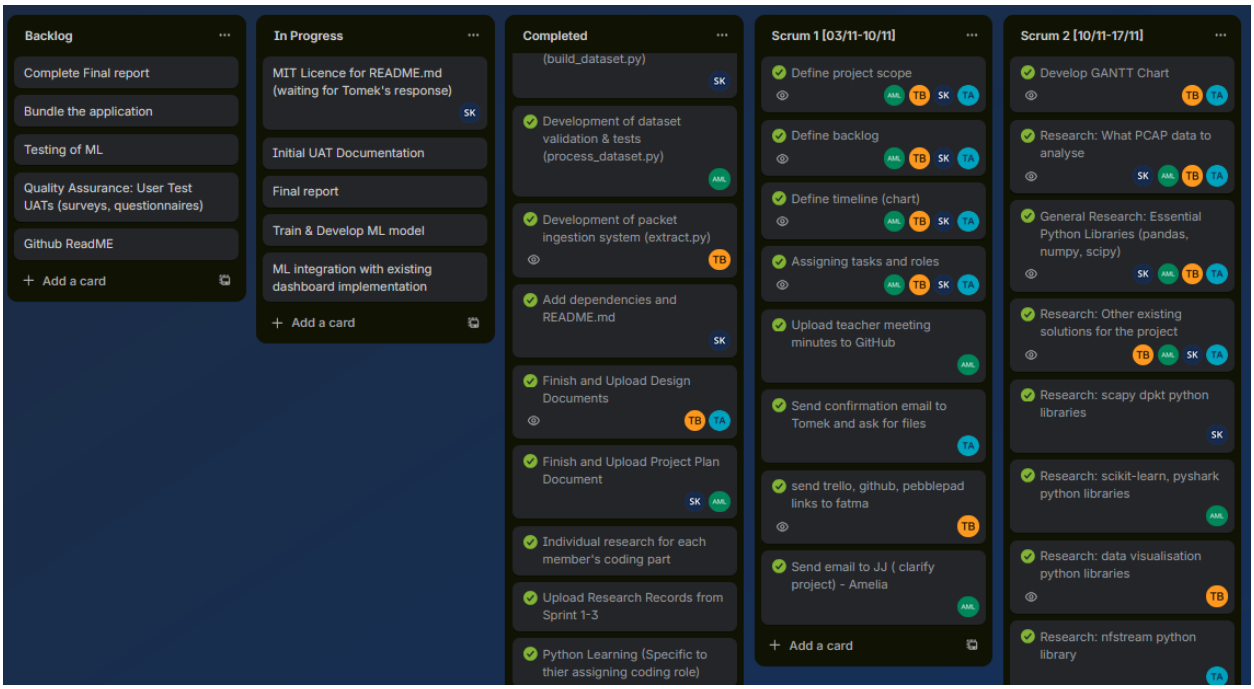


Figure 1 - Trello board snapshot

Risk assessment and mitigation (Sofia)

Project schedule (Tomiris)

The schedule was designed to ensure a logical progression from foundational system development to advanced machine learning integration, while maintaining alignment with client requirements and submission deadlines.

To support time management and task coordination, detailed Gantt charts were developed for each semester. These charts outlined task durations, sequencing, and key dependencies, enabling clear visualisation of the project timeline and ensuring transparency in workload allocation.

Traffic

Group 15 Project lead: Amelia

Project start: 11/3/2025

Display week: 1

SIMPLE GANTT CHART Template by Vertex42.com
https://www.vertex42.com/ExcelTemplates/simple-gantt-chart.html

TASK	ASSIGNED TO	PROGRESS	START	END
Sprint One: Initiation & Initial research				
Project scope analysis		100%	11/3/2025	11/5/2025
Definition of requirements		100%	11/6/2025	11/7/2025
Backlog analysis		100%	11/7/2025	11/9/2025
Tech stack & competition research		100%	11/10/2025	11/15/2025
Sprint Two: Development				
Create epics and user stories from requirements		100%	11/16/2025	11/18/2025
Create UML diagrams		100%	11/19/2025	11/21/2025
Implementation of PCAP file upload system		100%	11/23/2025	11/25/2025
Implement packet parsing		100%	11/25/2025	11/28/2025
Create dashboard design wireframes		100%	11/29/2025	11/30/2025
Sprint Three				
Implement dashboard visualisation		100%	12/3/2025	12/5/2025
Implement data chart visualization (traffic over time, protocol breai		100%	12/6/2025	12/13/2025
Completion of submission video		100%	12/14/2025	12/15/2025

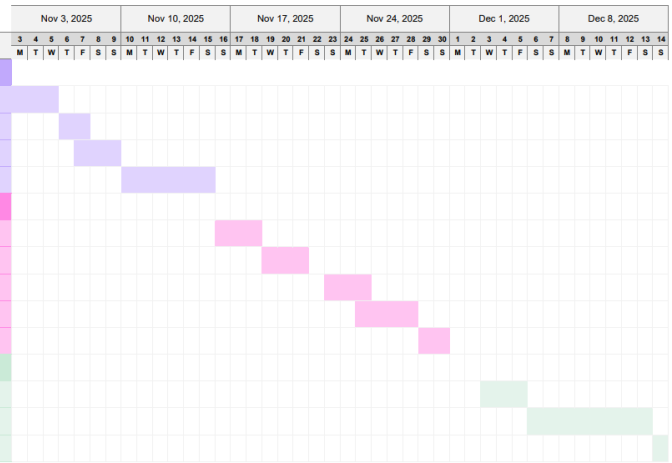


Figure 2 - Semester 1 Gantt Chart

Semester 1 focused on establishing the core system architecture and delivering a viable prototype.

- The first sprint of semester 1 was dedicated to defining the project scope, allocating team roles and conducting technical research related to network traffic analysis and PCAP data processing.
- The second sprint involved preparing the design documentation and project plan. During this phase, scripts were developed to extract and validate features from PCAP files, and the dataset was constructed.
- The third sprint was dedicated to implementing the dashboard interface and conducting unit testing to validate system functionality. This phase concluded with preparation for the interim submission milestone, ensuring that a functional prototype was delivered within the allocated timeframe.

All Semester 1 deliverables were completed according to the planned schedule.

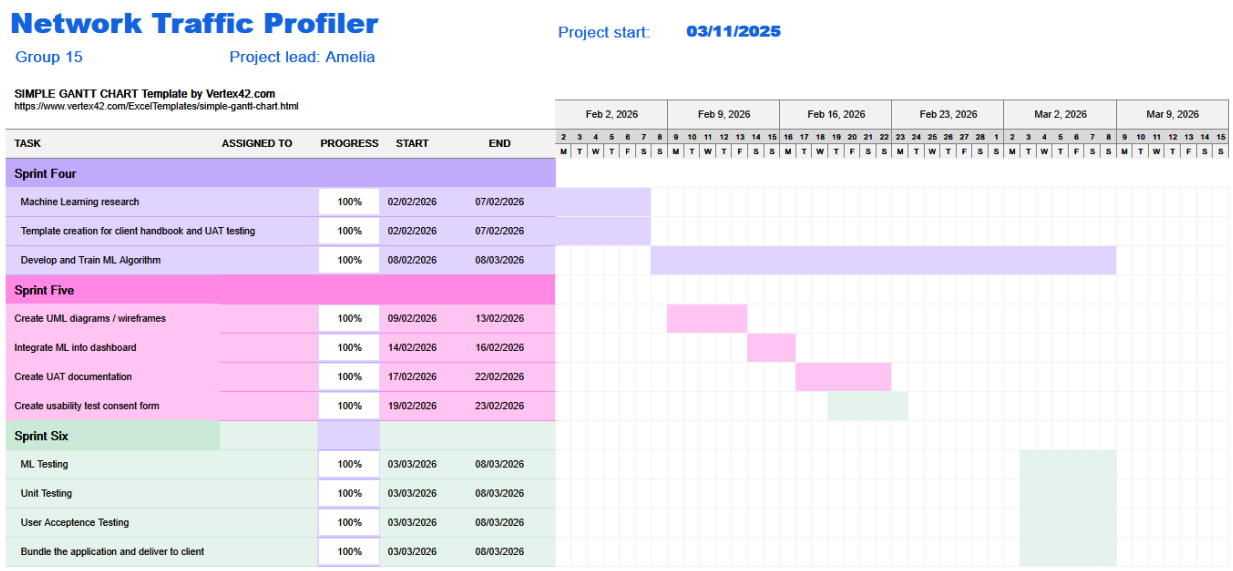


Figure 3 - Semester 2 Gantt Chart

Semester 2 shifted focus toward enhancing the system with machine learning capabilities and preparing the final deliverable.

- The first sprint involved research and selection of appropriate machine learning algorithms, alongside early preparation of final report documentation.
- The second sprint focused on updating UML diagrams to reflect architectural changes, developing and training the machine learning model using the prepared dataset, and integrating it into the existing dashboard framework.
- The final sprint was allocated to model evaluation, user acceptance testing (UAT), performance validation, and system refinement. Documentation was finalised in preparation for submission.

Project Milestones

Milestone	Description	Status
Viable Prototype Developed	Core system implementation and dashboard functional	Completed
Interim Submission	Submission of prototype and documentation	Completed
Machine Learning Integration	ML model developed and integrated into system	In Progress
Final Submission	Complete system, testing, and documentation	Pending

These milestones provided structured checkpoints that supported progress monitoring and accountability throughout the project lifecycle.

All Semester 1 deliverables were completed within their allocated timeframes, and no critical deadlines were missed. The phased structure of the schedule ensured a smooth transition from system development to machine learning integration, reducing rework and maintaining development momentum. The use of sprint-based planning supported incremental progress, clear task ownership, and continuous monitoring of deliverables.

Client management (Amelia)

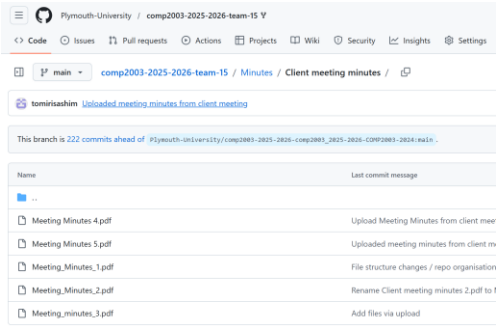
Effective client management was a key priority and a critical factor in the successful delivery of this project. It ensured continuous alignment with the client's requirements, expectations, and timelines. Maintaining a strong, collaborative relationship throughout the project lifecycle was achieved through structured communication, proactive feedback incorporation, and thorough documentation.

1. **Communication and Collaboration**
   Transparent and consistent communication channels were established to foster collaboration and ensure that the client remained fully informed of project progress:

- **Regular Meetings:**
  Bi-weekly meetings were conducted to review project status, discuss upcoming tasks, and address any client concerns. During the first semester, meetings were held every Thursday, and during the second semester, every Wednesday. Meetings took place either in person at **Portland Square, Room A03** or virtually via **Microsoft Teams**.

  Each session focused on:
  - Project timelines and milestone tracking
  - Progress updates on ongoing tasks
  - Planning for short-term and long-term objectives/tasks
  - Gathering client feedback
  - Demonstrating interim product features, including dashboards and AI model outputs
    [Image of meeting]

∉ **Meeting Records:**
All requirements, comments, and decisions discussed were formally recorded in client meeting minutes. This provided a transparent record of discussions and decisions, ensuring nothing was overlooked. These records can be accessed here.



- **Email Updates:**
  In addition to meetings, consistent email communications were used to share documentation, request approvals, and follow up on key decisions.

- **Interim Demonstrations:**
  Dashboard prototypes and interim versions of the product were presented to the client for review. This iterative demonstration approach ensured that the client's expectations were met at each stage, enabling early detection and correction of any discrepancies.

2. **Feedback and Change Management**
   Client feedback was actively incorporated into the development process to maintain alignment with project goals.

   This includes:
   - Adjusting dashboard visualizations based on client feedback, enhancing usability and clarity
   - Refining AI model output presentation according to client preferences
   - Prioritising requested features or modifications within sprint cycles

   Any suggested changes were formally recorded, evaluated for their impact on timeline and scope, and either approved or deferred based on mutual agreement. Continuous monitoring of deliverables against initial requirements helped manage expectations.

3. **Documentation and Approval**
   All major milestones, including project requirements, design choices and the final product, required client approval. Approvals were documented through emails, signed forms, or recorded meeting minutes. The client's formal sign off also ensured that the delivered dashboard met agreed specifications and expectations.

3) Detailed System Design (Tim & Tomiris)

– user story, ERD, prototype screenshots, HIPO chart, state diagrams, DFDs, sequence diagrams flowcharts, UML etc. Screenshots of snippets that you think are important can be placed here as well, but no need for full source code, just link to git repo.

Potential code snippets: Extract pcap data, ML training?

Epics and user stories

Once the initial requirements had been finalised, we mapped them into Epics which are broad categories of high level functionality. These were then further refined into User Stories which define features in an actionable format.

Epics and user stories

Epic One: PCAP Ingestion

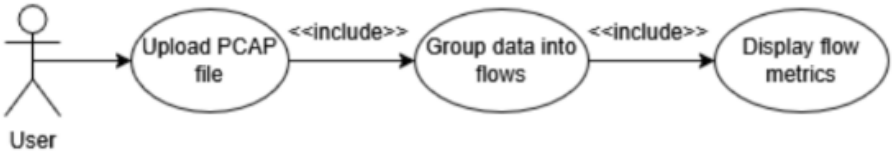
User stories

1. As a user I want to upload a PCAP file so that I can analyse my network traffic.
2. As a user I want to know if the analysis is still in progress, so I am aware the app is working.
3. As a user I want to be told if the PCAP file is corrupted or invalid so that the analysis fails safely.
4. As a **user** I want to be able to re-upload a new file so that I can analyse multiple datasets.

Epic Two: Data extraction and processing

User stories

1. As a user I want network traffic to be grouped into flows so that I can understand patterns.
2. As a user I want to view flow metrics so that I can interpret users' network behaviour.
3. As a user I want to be notified if my PCAP file contains no valid IP packet data.
4. As a user I want the system to be able to process large files without freezing.



Epic Three: Dashboard UI

User stories

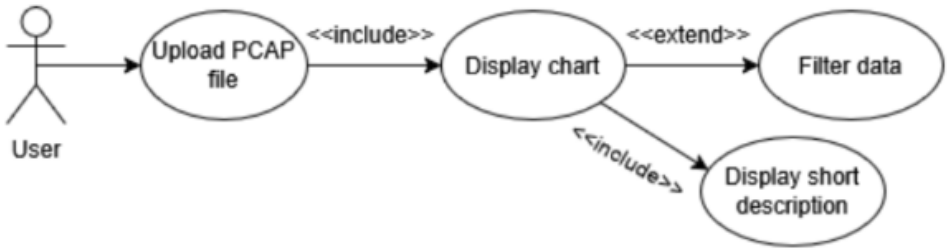
1. As a user I want a simple workflow so that I can easily analyse my data without advanced technical knowledge

2. As a user I want a clean and intuitive interface so that I can use the tool with minimal training
3. As a user I want the UI to function without crashing or freezing so that I can use the UI without hindrance.
4. As a user I want to see flow classifications in the dashboard so that I can understand traffic behaviour visually.
5. As a user I want anomalies to be highlighted in charts and tables so that I can easily spot them.
6. As a user I want to filter traffic by predicted class and anomaly status so that I can explore specific behaviours.

Epic Four: Data visualisation

User stories

1. As a user I want to be able to view interactive charts that show traffic size and flow statistics over time so that I can visually understand the data.
2. As a user I want to be able to filter the dashboard by protocol, time, and classification so that I can view specific activities.
3. As a user I want short descriptions for each chart so that I can understand what the data represents and means.
4. As a user I want summarised statistics so that I can make quick insights.



Epic Five: Documentation

User stories

1. As a user I want straightforward documentation so that I can understand how to get started.
2. As a user I want straightforward documentation of key network concepts so that I can understand the data and what it means
3. As a user I want documentation explaining how classifications are generated so that I understand system behaviour.
4. As a user I want documentation describing what the anomaly means and how it works so that I do not misinterpret results.

Epic Six: Code Maintainability

User stories

1. As a developer I want the code to be modular so that I can update individual components without breaking others.

Epic Seven: Performance and Error Handling

User stories:

1. As a developer I want data extraction and validation to be efficient so that large PCAP files do not incur long delays.
2. As a developer I want the dashboard application to display a readable, user-friendly error if the application crashes.
3. As a developer I want to conduct tests for each pipeline stage so that changes can be validated.

Epic Eight: User Action Classification (ML)

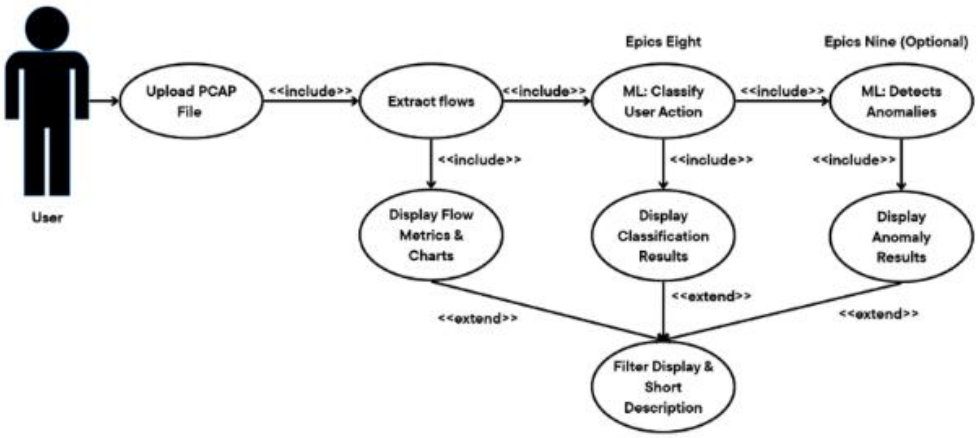
User Stories:

1. As a user I want network flows to be automatically classified so that I can understand user behaviour.
2. As a user I want each flow to have a label so that I can see what type of activity it represents.
3. As a user I want classification to run automatically after PCAP upload so that I do not need to trigger it manually.

Epic Nine: Anomaly Detection (ML – optional)

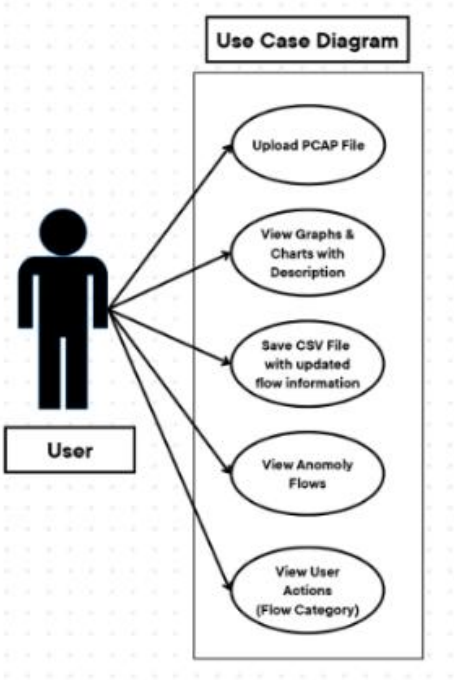
User Stories:

1. As a user I want anomalous flows to be clearly marked so that they stand out from normal traffic.
2. As a user I want the system to detect unusual or suspicious network behaviour so that I can identify potential threats or errors.



Epic Ten: ML Integration & Model

1. As a developer I want ML inference to run efficiently so that the dashboard updates without long delays.
2. As a developer I want to handle model errors gracefully so that the system does not crash if prediction fails.
3. As a developer I want to save and reuse trained models so that I do not need to retrain them every time.

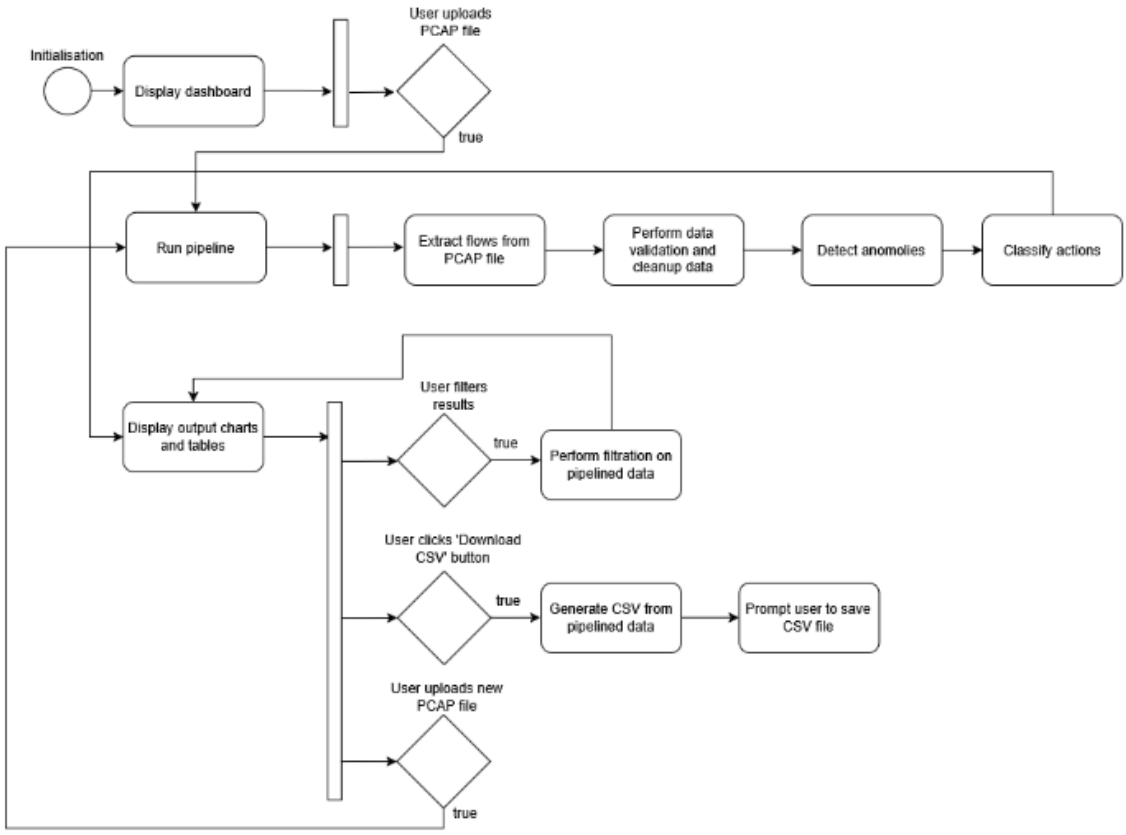


Wireframes (Tomi)

Activity diagram (Tim)

The activity diagram below shows the flow of the application from initialisation. The activities are split into three phases:

1. **Data ingestion –** The system shows the ‘home page’ of the dashboard until a valid PCAP file is uploaded.
2. **Processing pipeline –** Flows are extracted from the PCAP file, which are fed into a data validation function. ML anomaly detection is performed on the results, in addition to user activity classification.
3. **Interactive data display –** A state of interaction is entered where the user can browse, filter and export all the produced data.
    - **a.** A loop allows the raw pipeline data to be filtered endlessly without losing data.
    - **b.** A button allows the pipeline data (with any filters applied) to be downloaded to a CSV for offline exploration and data analysis.
    - **c.** The original PCAP file upload section remains visible and when a new file is uploaded the pipeline restarts.



Sequence Diagram (Tomi)

Prototype screenshots (Tim)

The following screenshots are from the initial prototype as of the 9th January 2026. At this point in time, about 80% of the final functionality was complete. Remaining efforts primarily involved developing machine learning powered user activity classification.

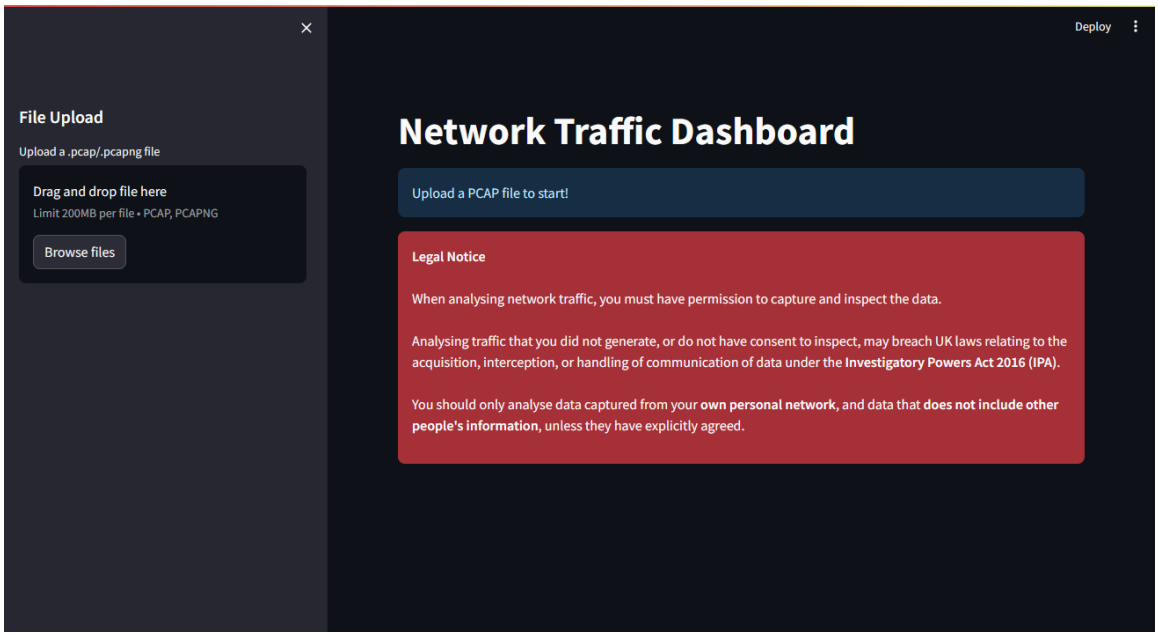


Figure 1 - Home page with PCAP upload section

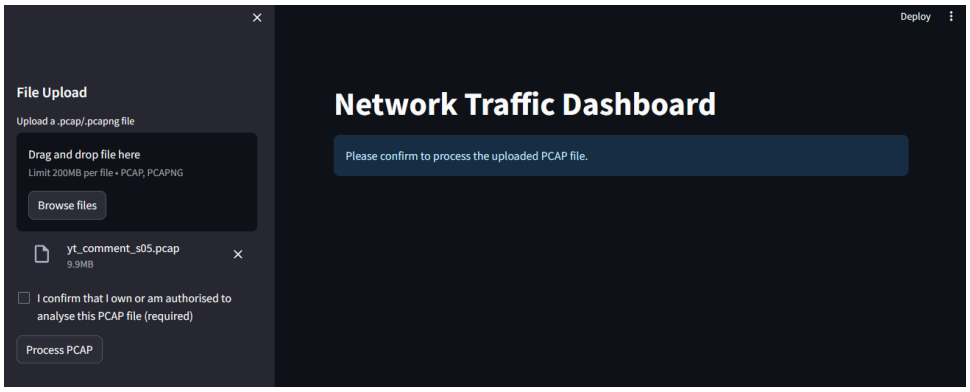


Figure 2 - Right to process prompt after uploading PCAP file

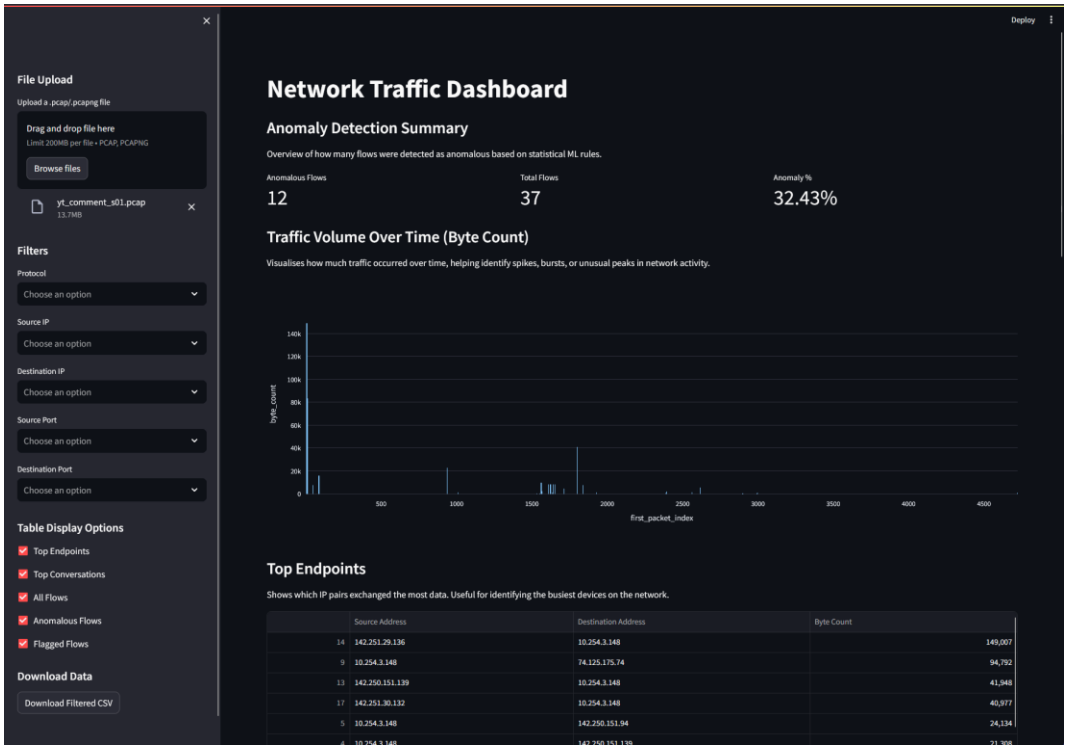


Figure 3 - Results of data pipeline, including interactable traffic volume graph

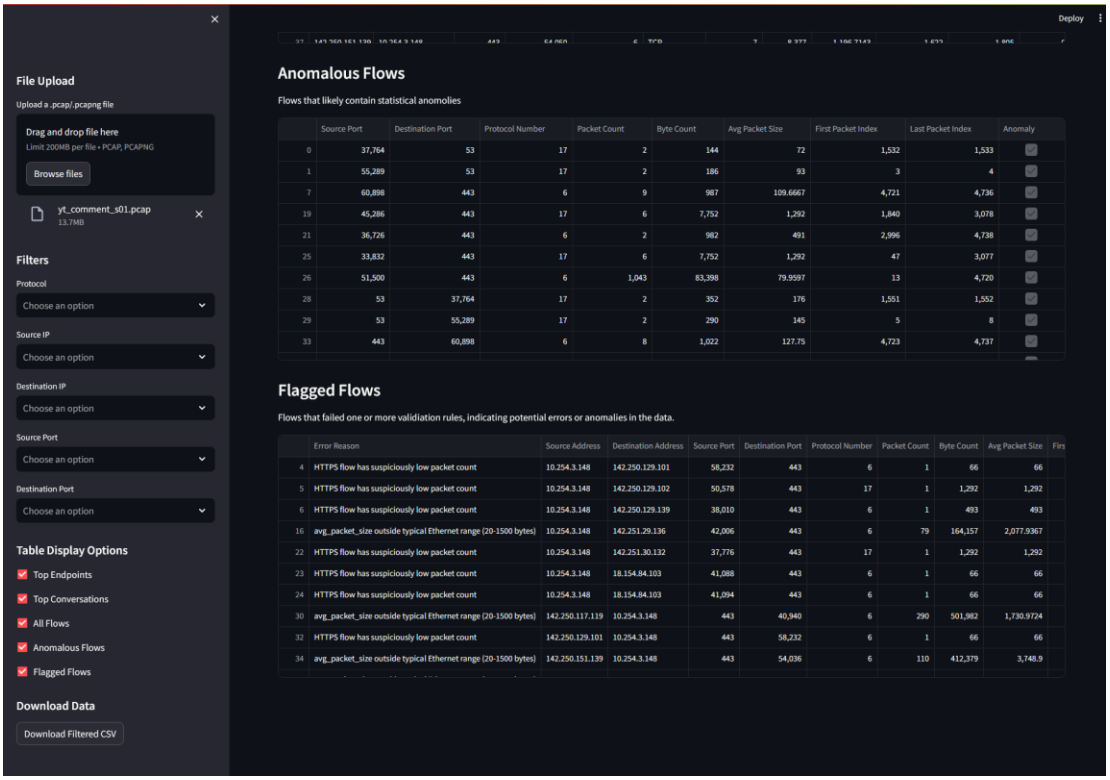


Figure 4 - Results of data pipeline, anomalous flows and flagged flows tables

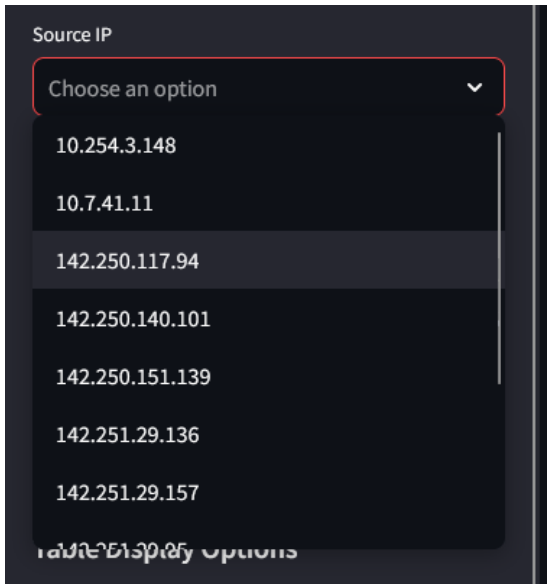


Figure 5 - Filtering data by source IP address

Class diagrams (Tomi)

Code snippets (Tim)

In this section we will discuss a few code snippets that provide vital functionality.

```
# Determine the action based off of the training folder and likely feature statistics
if action == "Play":
    # Usually the largest flow
    idx = df_flows["total_bytes"].idxmax()
    df_flows.loc[idx, "action"] = "Play"

elif action in ["Like", "Subscribe", "Comment", "Search"]:
    # Outbound heavy therefore picks the largest flow with an outbound ratio > 0.4
    candidates = df_flows[df_flows["outbound_ratio"] > 0.4]
    if not candidates.empty:
        idx = candidates["total_bytes"].idxmax()
        df_flows.loc[idx, "action"] = action
    else:
        # Fallback: just pick the largest if outbound ratio is weird
        idx = df_flows["total_bytes"].idxmax()
        df_flows.loc[idx, "action"] = action
```

HIPO diagram? (unsure if required)

DFD (unsure if required)

4) Quality Assurance (Sofia & Amelia)

– (post test: surveys & questionnaires), documentation on unit test ,UAT on methodology, results (Sofia)

- post-project support and any other quality assurance measures taken (Amelia)

Ensuring high-quality deliverables was a top priority throughout the development of the Network Traffic Profiler Dashboard. This was achieved through a combination of rigorous testing, structured reviews, and post-project support measures, maintaining reliability, usability, and alignment with the client's requirements.

Post- project Support

After the completion of the project, several support measures were planned to ensure smooth adoption, continued functionality, and the client's ability to maintain the dashboard independently:

Client Handover: A comprehensive walkthrough of the dashboard was conducted, demonstrating installation, configuration, operation, and troubleshooting procedures. This ensured the client could confidently use the dashboard in a production environment.

Documentation Provision:

The client was provided with a complete set of documentation, including:

- Up-to-date README files
- User manuals for end-user operations
- Design diagrams showing system architecture
- Annotated source code for clarity and maintainability
- Testing reports summarizing unit tests, feature validation and performance results

This comprehensive documentation ensures that both current users and future developers can understand, maintain, and extend the system as needed.

Knowledge Transfer: Detailed technical explanations and diagrams were shared to enable future developers or IT teams to integrate the dashboard into larger systems or perform further maintenance.

Other quality assurance measures taken
Several quality assurance practices were applied throughout the project lifecycle to maintain high standards:

Version Control and Peer Code Reviews:
GitHub was used to track all code changes. Team members reviewed each other's code to prevent errors and ensure consistent code quality. Clear commit messages and structured branching improved traceability.

Continuous Testing:
Beyond unit tests, manual testing of dashboard functions, including uploads, interactive charts, tables and ML model outputs, was conducted to validate real-world performance and ensure accuracy and robustness.

5) Conclusion (Tim, Tomiris)

– summarize your project achievements and merits.

6) References (3 References Each)

– any referenced sites/papers you cited within your report. Recommended 10-15 References.

Kumar, G. and Bhati, P.K. (2012) 'Impact of Agile Methodology on Software Development Process,' International Journal of Computer Technology and Electronics Engineeri, 2(4). https://www.researchgate.net/profile/Gaurav-Kumar-175/publication/255707851_Impact_of_Agile_Methodology_on_Software_Development_Process/links/00b49520489442e12d000000/Impact-of-Agile-Methodology-on-Software-Development-Process.pdf.

7) Appendix

– attach the 1000 word CW Submission Evidence and Evaluation file for each member as appendix. This will not count towards the page limit.